# 🥭 MNGO: Definitive Domain Architecture (100 Core Modules)

MNGO is an ultra-scalable, modular **Venue Operating System (VOS)** designed for high-concurrency environments like theme parks, resorts, and multi-venue entertainment complexes. The system is organized into **11 specialized domains**, ensuring strict isolation, logical clarity, and high-performance execution of core business logic.

---

## 🔐 IDENTITY DOMAIN ( `packages/Mngo/Identity/` )

The **Identity Triad** (User, Actor, Holder) and the security backbone of the ecosystem.

### 1. Auth ( `mngo/auth` )

- **Description**: Centralized authentication bridge and token management system.
- **Features**: JWT issuance, OAuth2 provider integration, session management, and cross-domain token validation.
- **Capabilities**: High-speed token verification, multi-driver session storage (Redis/Database), and anonymous guest token generation.
- **Boundaries**: Handles *credentials* and *tokens*; does NOT handle profile data or permissions (deferred to User and RBAC).
- **How it Works**: Intercepts requests internally to validate JWTs against the central auth store, providing a unified `AuthResult`.
- **Use Cases**: Guest login via mobile app, staff login to POS, and IoT device authentication.
- **Integrations**: `User` (for credential lookup), `Actor` (for context binding), `ApiGateway` (for perimeter security).

### 2. User ( `mngo/user` )

- **Description**: Management of the authentication lifecycle and security credentials.
- **Features**: Multi-factor authentication (MFA), Single Sign-On (SSO), password policy enforcement, and account recovery.
- **Capabilities**: Support for biometric hashes, social login linking, and account locking/unlocking.
- **Boundaries**: Handles the *account* and *security metadata*; does NOT handle physical attributes (deferred to Holder).

- **How it Works**: Acts as the primary repository for login identities across various platforms (Staff/Guest).
- **Use Cases**: Changing a password, enabling 2FA, or linking a Google account to a guest profile.
- **Integrations**: `Auth` (for token generation), `Holder` (linked via ID), `Notification` (for MFA codes).

## 3. Actor ( `mngo/actor` )

- **Description**: Unified execution context for every request, identifying *who* or *what* is performing an action.
- **Features**: Context resolution (User vs. Guest vs. Device vs. System), actor-user mapping, and ephemeral identity management.
- **Capabilities**: Dynamic resolution of "Effective Identity" during a request lifecycle.
- **Boundaries**: Focuses on the *execution entity*; does NOT manage persistent profile data.
- **How it Works**: Decorates the request with an `ActorInfo` object that tells other modules who is the current initiator.
- **Use Cases**: Distinguishing between an action taken by a guest via an app vs. a staff member overriding a price at POS.
- **Integrations**: `Auth` (context extraction), `Audit` (logging the actor).

## 4. Holder ( `mngo/holder` )

- **Description**: The persistent system of record for physical entities (Guests, Staff, Business partners).
- **Features**: Profile management, address book, contact verification, and immutable unique identifiers (UUIDs).
- **Capabilities**: Dynamic attribute storage (JSON-B), cross-venue profile linking, and KYC/Identity document storage.
- **Boundaries**: Handles the *physical identity* and *data*; does NOT handle authentication or active permissions.
- **How it Works**: Serves as the global profile hub, ensuring a person remains the same entity across different venues.
- **Use Cases**: Updating a shipping address, storing a guest's birthday, or uploading a staff member's ID photo.
- **Integrations**: `User` (account link), `CRM` (for behavioral analysis), `Geography` (address validation).

## 5. RBAC ( `mngo/rbac` )

- **Description**: Hierarchical Role-Based Access Control system.

- **Features**: Role nesting, permission assignment, inheritance logic, and granular scope management.
- **Capabilities**: Efficient role evaluation via adjacency lists, mass-permission updates, and venue-scoped roles.
- **Boundaries**: Manages *roles* and *static permissions*; does NOT handle dynamic runtime policies (deferred to Policy Engine).
- **How it Works**: Maps Actors to Roles, and Roles to Permissions, providing a binary `can()` response.
- **Use Cases**: Assigning a "Shift Supervisor" role which inherits permissions from the "Cashier" role.
- **Integrations**: `Identity` (all modules), `Governance` (for scoping permissions to venues).

## 6. Policy Engine ( `mngo/policy` )

- **Description**: Dynamic Attribute-Based Access Control (ABAC) for complex business rules.
- **Features**: Runtime rule evaluation, context-aware decisions (time-based, location-based), and complex condition trees.
- **Capabilities**: High-performance JSON-B rule parsing and asynchronous policy checking for non-critical flows.
- **Boundaries**: Evaluates *dynamic conditions*; does NOT store the roles themselves (uses RBAC output as input).
- **How it Works**: Evaluates a request against a set of policies: e.g., "Allow refund only if staff is in the same venue and shift is open."
- **Use Cases**: Restricting scanner access to specific dates or allowing discounts only to premium members.
- **Integrations**: `RBAC` , `Governance` (location context), `Events` (time context).

## 7. Approval & Escalation ( `mngo/approval` )

- **Description**: Centralized workflow management for actions requiring management intervention.
- **Features**: Multi-stage approval paths, dynamic escalation triggers, and digital signature logs.
- **Capabilities**: Asynchronous notifications for approvals and automated fallback logic if a manager is offline.
- **Boundaries**: Handles the *workflow state*; does NOT execute the actual business logic being approved.
- **How it Works**: Intercepts restricted actions and creates an `ApprovalRequest` that must be signed before the action proceeds.
- **Use Cases**: Approving a refund over $500 or a staff roster change.
- **Integrations**: `Identity` (for signatories), `Notification` (alerts), `Finance` / `Operations` .

## 8. Audit & Logging ( `mngo/audit` )

- **Description**: Immutable digital record of every sensitive action and system state change.
- **Features**: Event sourcing logs, tamper-evident audit trails, and data change tracking (before/after).
- **Capabilities**: High-volume log ingestion via asynchronous queues and log rotation strategies for legal compliance.
- **Boundaries**: Records *historical events*; does NOT handle real-time monitoring (deferred to Telemetry).
- **How it Works**: Captures `Actor`, `Action`, `Timestamp`, and `Context` for every critical system interaction.
- **Use Cases**: Investigating a suspicious transaction or auditing who changed a price setting.
- **Integrations**: Entire ecosystem.

---

# 🏢 GOVERNANCE DOMAIN (`packages/Mngo/Governance/`)

The organizational hierarchy and multi-tenant management layer.

## 9. Tenant (`mngo/tenant`)

- **Description**: High-level isolation for different business entities or platform owners.
- **Features**: Resource isolation, branding configuration, and global platform settings.
- **Capabilities**: Database schema isolation (multi-db or shared-db) and tenant-specific API rate limits.
- **Boundaries**: Manages *global isolation*; does NOT manage physical venue logic.
- **How it Works**: Identifies the tenant from the host or header and sets the global execution scope.
- **Use Cases**: Hosting multiple independent theme parks on a single MNGO instance.
- **Integrations**: Entire ecosystem (via `Identity` context).

## 10. Branch (`mngo/branch`)

- **Description**: Logical business divisions or regional headquarters.
- **Features**: P&L tracking per branch, regional management hierarchy, and resource consolidation.
- **Capabilities**: Multi-venue aggregation for reporting and centralized procurement logic.
- **Boundaries**: Represents a *business division*; does NOT represent physical coordinates (deferred to Venue).
- **How it Works**: Groups multiple Venues under a single management and accounting umbrella.
- **Use Cases**: Managing "MNGO Parks Middle East" as a branch that covers several physical parks.
- **Integrations**: `Venue`, `Finance` (settlements), `Reporting`.

## 11. Venue ( `mngo/venue` )

- **Description**: Detailed management of physical venue operations.
- **Features**: Operating hours, physical zones/sectors, capacity limits, and venue-specific rules.
- **Capabilities**: Physical location mapping, real-time venue status (Open/Closed), and guest-to-venue binding.
- **Boundaries**: Manages the *individual physical entity*; does NOT handle business hierarchy (deferred to Branch).
- **How it Works**: Serves as the central registry for all physical locations where business is conducted.
- **Use Cases**: Setting the opening time for a specific water park.
- **Integrations**: `Department` , `Device` , `Operations` .

## 12. Region (Business) ( `mngo/region` )

- **Description**: Jurisdictional boundaries and business regions.
- **Features**: Regional tax jurisdiction, compliance rules, and currency settings.
- **Capabilities**: Automatic determination of applicable laws and taxes based on venue region.
- **Boundaries**: Handles *legal and business regions*; does NOT handle geography (countries/cities, see Ecosystem).
- **How it Works**: Maps a Venue to a Business Region to apply correct financial and legal logic.
- **Use Cases**: Ensuring a park in Dubai follows VAT laws while a park in Riyadh follows KSA tax rules.
- **Integrations**: `Geography` , `Finance/Tax` , `Legal` .

## 13. Department ( `mngo/department` )

- **Description**: Functional business units within a venue.
- **Features**: Cost center tracking, staff assignment, and unit-specific inventory management.
- **Capabilities**: Independent operational controls for F&B, Retail, and Attractions within a single venue.
- **Boundaries**: Represents a *functional unit*; does NOT manage the physical building itself.
- **How it Works**: Partitions a Venue into manageable operational segments (e.g., "Gift Shop A", "Main Kitchen").
- **Use Cases**: Running a specific P&L report just for the "Front Gate F&B" department.
- **Integrations**: `Venue` , `Rostering` , `Finance` .

---

[TO BE CONTINUED: FINANCE, TICKETING, EVENTS...]

---

# 💰 FINANCE DOMAIN ( `packages/Mngo/Finance/` )

The financial source of truth, managing all money movement, tax compliance, and revenue recognition.

## 14. Payment ( `mngo/payment` )

- **Description**: Gateway orchestration and payment lifecycle management.
- **Features**: Multi-gateway routing, partial captures, payment intent management, and 3D-Secure handling.
- **Capabilities**: Support for global providers (Stripe, Adyen) and local methods (Mada, ApplePay).
- **Boundaries**: Handles the *act of paying*; does NOT manage order logic (deferred to Order/Cart).
- **How it Works**: Standardizes various payment gateway responses into a unified `TransactionResult`.
- **Use Cases**: A guest paying for tickets with a saved credit card or Apple Pay.
- **Integrations**: `Transaction`, `Receipt`, `Ecosystem/Integration`.

## 15. Payouts ( `mngo/payout` )

- **Description**: Management of disbursements to external partners, vendors, and staff.
- **Features**: Scheduled payouts, vendor commission splitting, and bank detail management.
- **Capabilities**: Bulk payout processing and integration with payout providers (Stripe Connect).
- **Boundaries**: Handles *outgoing funds*; does NOT handle internal payroll (deferred to HR/Rostering).
- **How it Works**: Calculates net amounts due to partners after commissions and fees are deducted.
- **Use Cases**: Paying out an external food vendor for their weekend sales at the venue.
- **Integrations**: `Settlement`, `B2B`, `Ecosystem/Integration`.

## 16. Settlement ( `mngo/settlement` )

- **Description**: High-level reconciliation of gateway captures against bank deposits.
- **Features**: Daily reconciliation reports, discrepancies detection, and automated bank feed matching.
- **Capabilities**: Support for various bank statement formats (MT940, BAI2) and automated GL posting.
- **Boundaries**: Handles *financial consistency*; does NOT handle real-time payment auth.
- **How it Works**: Compares the system's `Transaction` log with external financial statements to verify every cent.
- **Use Cases**: Finance team verifying that the $50k in Stripe captures matches the bank deposit.
- **Integrations**: `Transaction`, `Payment`, `Payout`.

## 17. Refunds ( `mngo/refund` )

- **Description**: Centralized logic for reversing financial transactions.
- **Features**: Partial/Full refunds, refund reason tracking, and multi-stage approval for large amounts.
- **Capabilities**: Automated reversal through original payment gateways and integration with credit notes.
- **Boundaries**: Handles the *reversal of funds*; does NOT manage the "return" of physical goods (Operations).
- **How it Works**: Links back to the original `Payment` and issues a `RefundIntent` to the gateway.
- **Use Cases**: A guest canceling a ticket 24 hours in advance and receiving a full refund.
- **Integrations**: `Payment`, `Transaction`, `Approval`.

## 18. Chargebacks ( `mngo/chargeback` )

- **Description**: Management of forced financial reversals initiated by banks.
- **Features**: Dispute evidence collection, deadline tracking, and automated response workflows.
- **Capabilities**: Integration with gateway APIs to receive real-time chargeback alerts.
- **Boundaries**: Handles the *dispute lifecycle*; does NOT handle the initial payment logic.
- **How it Works**: Tracks the status of a dispute from `Received` to `Won` or `Lost`.
- **Use Cases**: Responding to a guest who claims they didn't authorize a $100 ticket purchase.
- **Integrations**: `Payment`, `Audit`, `Notification`.

## 19. Transaction ( `mngo/transaction` )

- **Description**: The immutable financial ledger of all activities.
- **Features**: Double-entry bookkeeping support, immutable sequence IDs, and financial metadata storage.
- **Capabilities**: High-speed ledger ingestion and queryable historical financial data.
- **Boundaries**: Records *what happened*; does NOT implement the business logic of *why*.
- **How it Works**: Every financial act in MNGO must emit a `Transaction` record for auditability.
- **Use Cases**: Generating a detailed financial audit log for the end-of-year tax review.
- **Integrations**: Entire Finance domain.

## 20. Invoice ( `mngo/invoice` )

- **Description**: Generation and management of formal billing documents.
- **Features**: PDF generation, recurring invoice scheduling, and partial billing support.
- **Capabilities**: Customizable templates per venue and support for multi-currency invoicing.

- **Boundaries**: Handles the *document*; does NOT handle the payment (Payment) or the ledger entry (Transaction).
- **How it Works**: Aggregates line items from Sales/Order into a legally compliant PDF document.
- **Use Cases**: Issuing a monthly invoice to a corporate client for group booking events.
- **Integrations**: `Order`, `B2B`, `Geography/Tax`.

## 21. Credit Memo ( `mngo/credit-memo` )

- **Description**: Processing of formal documents that reverse an invoice (Credit Notes).
- **Features**: Linked credit issuance, partial memo support, and inventory re-entry triggers.
- **Capabilities**: Formal record keeping for balance adjustments without direct cash refunds.
- **Boundaries**: Handles the *formal adjustment document*; does NOT handle cash movement.
- **How it Works**: Adjusts the customer's balance or offsets an existing unpaid invoice.
- **Use Cases**: Issuing a $50 credit for a guest who experienced a ride breakdown.
- **Integrations**: `Invoice`, `Refund`, `Wallet`.

## 22. Receipt ( `mngo/receipt` )

- **Description**: Guest-facing proof of purchase (VAT compliant).
- **Features**: Digital receipting (Email/SMS), thermal printer support (POS), and QR-code verification.
- **Capabilities**: High-speed receipt generation for high-traffic POS terminals.
- **Boundaries**: Handles *consumer proof*; does NOT handle formal B2B invoicing rules.
- **How it Works**: Simple, optimized snapshot of a completed transaction for immediate guest delivery.
- **Use Cases**: Printing a small slip at a quick-service food counter after a guest buys a soda.
- **Integrations**: `Order`, `Terminal`, `Notification`.

## 23. Wallet (EFC) ( `mngo/wallet` )

- **Description**: Stored-value accounts for guests and staff (Electronic Fund Container).
- **Features**: Balance management, bonus points, family shared wallets, and tiered credit limits.
- **Capabilities**: Offline balance caching for ultra-low latency spending (E Ecosystem).
- **Boundaries**: Handles the *virtual balance*; does NOT handle the initial loading of cash (Payment).
- **How it Works**: Operates as a "private ledger" where guests can spend pre-loaded credits instantly.
- **Use Cases**: A child using their wristband to buy a snack using pre-loaded funds from their parent's wallet.
- **Integrations**: `Payment`, `Cart`, `Ticketing`.

## 24. Tax Engine ( `mngo/tax` )

- **Description**: Standalone regional tax calculation and compliance service.
- **Features**: VAT/GST calculation, tax-exempt rule management, and tax reporting.
- **Capabilities**: Integration with external tax compliance services (e.g., Avalara) where needed.
- **Boundaries**: Calculates *amounts*; does NOT handle the actual invoicing (deferred to Invoice).
- **How it Works**: Takes a subtotal and a location/category and returns the dynamic tax breakdown.
- **Use Cases**: Applying 15% VAT to a ticket sale in Saudi Arabia but 5% VAT in the UAE.
- **Integrations**: `Order`, `Invoice`, `Region`.

## 25. Fee Engine ( `mngo/fee` )

- **Description**: Management of service, booking, and convenience fees.
- **Features**: Flat-fee, percentage-fee, and tiered-fee logic based on channel or product.
- **Capabilities**: Dynamic fee application during checkout based on guest segments.
- **Boundaries**: Manages *additional costs*; does NOT handle the core product price (Pricing).
- **How it Works**: Injects fee line-items into a Cart or Order before final calculation.
- **Use Cases**: Adding a $2 "Convenience Fee" for tickets purchased via the mobile app vs. the box office.
- **Integrations**: `Cart`, `Order`, `Pricing`.

## 26. Subscription ( `mngo/subscription` )

- **Description**: Lifecycle management for recurring billing and plans.
- **Features**: Annual passes, monthly memberships, trial periods, and auto-renewal logic.
- **Capabilities**: Handling dunning (failed payment retries) and plan upgrades/downgrades.
- **Boundaries**: Manages the *plan lifecycle*; uses the Payment module for the actual monthly charge.
- **How it Works**: Tracks the validity dates of a subscription and triggers renewal events.
- **Use Cases**: A guest paying $50/month for an "Unlimited Entry" season pass.
- **Integrations**: `Payment`, `Entitlement`, `Order`.

## 27. Gift Card ( `mngo/gift-card` )

- **Description**: Management of physical and digital gift cards.
- **Features**: Card activation, balance top-ups, expiry management, and security PIN verification.
- **Capabilities**: Integration with physical card encoders and digital code generation for emails.
- **Boundaries**: Handles the *card entity*; the balance itself is often a specific entry in the Wallet system.
- **How it Works**: Maps a unique card/code to a secure balance that can be redeemed at checkout.
- **Use Cases**: Buying a $100 gift card for a friend to use anywhere in the resort.
- **Integrations**: `Wallet`, `Payment`, `Order`.

---

# 🎟 TICKETING DOMAIN (`packages/Mngo/Ticketing/`)

The core entitlement engine for the MNGO Venue Operating System.

## 28. UVTI (`mngo/uvti`)

- **Description**: Universal Virtual Ticket Identity – the central container for all entitlements.
- **Features**: Unique ticket ID generation, multi-entitlement packing, and state tracking (Ready/Used/Shared).
- **Capabilities**: Secure hashing for third-party verification and support for complex metadata.
- **Boundaries**: Handles the *identity of the ticket*; does NOT handle the physical media (Media Management).
- **How it Works**: Every sale of an entitlement results in the generation of a UVTI that ties everything together.
- **Use Cases**: A single QR code that represents park entry, a lunch voucher, and a fast-pass.
- **Integrations**: `Product`, `Entitlement`, `Media`.

## 29. Product (`mngo/product`)

- **Description**: Catalog management for all ticket-based products and SKUs.
- **Features**: General Admission (GA), Time-Slot tickets, Dated tickets, and Package bundles.
- **Capabilities**: Support for dynamic inventory attributes and multi-language descriptions.
- **Boundaries**: Manages the *commercial definition*; does NOT handle real-time pricing (Pricing).
- **How it Works**: Defines *what* is for sale and its intrinsic properties (validity, constraints).
- **Use Cases**: Defining a "Weekend Early Bird Ticket" vs. a "VIP After Dark Pass".
- **Integrations**: `Pricing`, `Order`, `DAM`.

## 30. Pricing (`mngo/pricing`)

- **Description**: Centralized pricing settings, rule management, and manual overrides.
- **Features**: Base price management, rule-based adjustments (e.g., student discount), and manual price sheets.
- **Capabilities**: Highly performant price lookup engine cached at the edge.
- **Boundaries**: Manages *configured pricing*; does NOT calculate AI recommendations (deferred to Price Engine).
- **How it Works**: Retrieves the correct price for a product given the current actor, time, and channel.
- **Use Cases**: Setting a $99 price for adults and $79 for children for the summer season.
- **Integrations**: `Product`, `Intelligence/PriceEngine`.

## 31. Seatmap ( `mngo/seatmap` )

- **Description**: Visual seating and inventory allocation engine.
- **Features**: Interactive seat selection, zone management (VIP/Economy), and dynamic seat locking.
- **Capabilities**: Support for complex SVG seatmaps and high-concurrency "holding" logic.
- **Boundaries**: Manages the *physical layout and availability*; does NOT handle the financial transaction.
- **How it Works**: Tracks the occupancy state of every individual seat or space in a venue.
- **Use Cases**: A guest choosing their specific seat for a live theater performance within the park.
- **Integrations**: `Product` , `Order` , `Operations` .

## 32. AccessControl ( `mngo/access-control` )

- **Description**: Real-time gate validation and scanning orchestration.
- **Features**: Scanner communication, anti-passback rules, and dynamic gate assignment.
- **Capabilities**: Ultra-low latency validation (<200ms) and support for offline-first gate operation.
- **Boundaries**: Handles the *validation logic*; does NOT manage the physical scanner hardware (Device).
- **How it Works**: Receives a `ScanEvent` , matches it to a `UVTI` , and returns a `Grant` or `Deny` signal.
- **Use Cases**: Scanning a wristband at the park entrance to verify entry validity for that day.
- **Integrations**: `UVTI` , `Entitlement` , `Device` .

## 33. Entitlement ( `mngo/entitlement` )

- **Description**: The "Usage Logic" engine that tracks limits and quotas.
- **Features**: Count-based usage (e.g., 5 rides), time-based usage (e.g., all day), and specific exclusion rules.
- **Capabilities**: Real-time incrementing and decrementing of usage counts during a visit.
- **Boundaries**: Manages the *rights and counts*; does NOT handle the identity of the guest.
- **How it Works**: Defines the "rules of use" for a `UVTI` (e.g., "Valid once for the Log Flume").
- **Use Cases**: Ensuring a "Quick Pass" is only used for the 3 rides the guest paid for.
- **Integrations**: `UVTI` , `AccessControl` , `Subscription` .

## 34. Media Management ( `mngo/media` )

- **Description**: Management of physical and digital carrier media (NFC, RFID, QR).
- **Features**: Wristband pairing, RFID encoding, QR code generation, and biometric link support.
- **Capabilities**: Multi-media linking (one UVTI can be on a wristband AND an app).
- **Boundaries**: Handles the *carrier*; does NOT handle the underlying entitlement logic.

- **How it Works**: Maps a physical ID (e.g., NFC UID) to a `UVTI` or a `Holder`.
- **Use Cases**: Pairing a silicon wristband to a guest's entrance ticket at the welcome desk.
- **Integrations**: `UVTI`, `Holder`, `Device`.

### 35. Photo Engine ( `mngo/photo` )

- **Description**: Commercial ride-photo and event-photo linking system.
- **Features**: Automatic facial/RFID matching, digital photo delivery, and photo pass commercialization.
- **Capabilities**: Integration with physical ride cameras and watermark/processing logic.
- **Boundaries**: Handles *commercial pairing and delivery*; does NOT handle image asset storage (deferred to DAM).
- **How it Works**: Matches a timestamped ride capture with a guest's position/RFID scan.
- **Use Cases**: A guest scanning their app at a kiosk to view and buy photos taken during a roller coaster ride.
- **Integrations**: `Ecosystem/DAM`, `UVTI`, `Device`.

---

---

## 🎭 EVENTS DOMAIN ( `packages/Mngo/Events/` )

Orchestration of time-based experiences, capacity management, and guest flow.

### 36. Event (Lifecycle) ( `mngo/event` )

- **Description**: Management of the scheduling and lifecycle of venue activities.
- **Features**: Event scheduling, capacity mapping, calendar management, and session control.
- **Capabilities**: Support for recurring events, multi-session activities, and real-time availability checks.
- **Boundaries**: Handles the *meta-data and schedule*; does NOT handle the sale (deferred to Order).
- **How it Works**: Serves as the master calendar for all bookable experiences within a venue.
- **Use Cases**: Scheduling a daily "Dolphin Show" with three sessions and a capacity of 500 per session.
- **Integrations**: `Product`, `Resource`, `Operations`.

### 37. Virtual Queue ( `mngo/queue` )

- **Description**: Wait-time management and digital queueing orchestration.
- **Features**: Wait-time prediction, virtual queue entry, and notification triggers for guest re-entry.
- **Capabilities**: AI-driven predictive modeling for wait times and load balancing across attractions.

- **Boundaries**: Handles the *queue state*; does NOT manage physical scanning at the entrance.
- **How it Works**: Tracks throughput and current wait times to provide guests with a "Virtual Spot".
- **Use Cases**: A guest joining a virtual queue for a roller coaster to avoid standing in a physical line.
- **Integrations**: `AccessControl`, `Notification`, `Intelligence/AI`.

### 38. Resource Management ( `mngo/resource` )

- **Description**: Planning and allocation of physical spaces and staff assets.
- **Features**: Room booking, equipment assignment, and space utilization tracking.
- **Capabilities**: Prevention of resource over-allocation and support for maintenance-driven downtime.
- **Boundaries**: Manages *entities*; does NOT handle staff labor laws (deferred to Rostering).
- **How it Works**: Maps event requirements to available assets (e.g., "Mascot Suit A" or "Theater Room B").
- **Use Cases**: Reserving a private party room for a birthday event within the venue.
- **Integrations**: `Event`, `Maintenance`, `Rostering`.

---

## 🛒 POS DOMAIN ( `packages/Mngo/POS/` )

The front-line operational interface for staff and terminal-based transactions.

### 39. Terminal ( `mngo/terminal` )

- **Description**: Logical and physical management of service points.
- **Features**: IP-to-terminal mapping, hardware peripheral routing (Printers/Scanners), and station modes.
- **Capabilities**: Remote terminal configuration and hardware health monitoring.
- **Boundaries**: Handles the *station logic*; does NOT handle the sale content (deferred to Sales domain).
- **How it Works**: Identifies the source of a transaction and routes printing or scanning events accordingly.
- **Use Cases**: Configuring a POS terminal to be a "Food Kiosk" vs. a "Ticket Window".
- **Integrations**: `Device`, `Cashiering`, `Sales/F&B`.

### 40. Cashiering ( `mngo/cashiering` )

- **Description**: Financial lifecycle of a staff member's drawer or float.
- **Features**: Float assignment, blind counts, cash-out reports, and shortage/overage tracking.
- **Capabilities**: Support for multi-currency cash handling and automated Z-reporting.

- **Boundaries**: Handles *physical money and drawer state*; does NOT handle JWT auth (Auth).
- **How it Works**: Tracks every physical currency movement associated with a specific staff session.
- **Use Cases**: A cashier performing a "Mid-Day Drop" to clear excess cash from their drawer.
- **Integrations**: `Terminal`, `Transaction`, `Shift`.

## 41. Shift Management ( `mngo/shift` )

- **Description**: Management of staff operational sessions.
- **Features**: Shift clock-in/out, temporal handover notes, and session productivity tracking.
- **Capabilities**: Automated shift termination (graceful close) and integration with labor compliance rules.
- **Boundaries**: Handles the *operational session*; does NOT manage the HR payroll roster (Rostering).
- **How it Works**: Groups cashiering and terminal activities into a bounded "Shift" for reporting.
- **Use Cases**: A manager reviewing the total sales performance of the "Morning Shift".
- **Integrations**: `Cashiering`, `Actor`, `Operations/Rostering`.

## 42. Offline Sync ( `mngo/sync` )

- **Description**: Resilience layer for edge operations during network instability.
- **Features**: Local-first data caching, background synchronization, and conflict resolution.
- **Capabilities**: Support for "Sales Pending Sync" status and robust delta-only updates.
- **Boundaries**: Handles *data transport*; does NOT handle the core business rules (executed locally or via proxy).
- **How it Works**: Uses an edge cache (e.g., SQLite/Redis) to allow POS operation without a cloud connection.
- **Use Cases**: Continuing to sell sodas at a remote kiosk when the park Wi-Fi goes down.
- **Integrations**: `ApiGateway`, `Order`, `Terminal`.

## 43. POS Interface ( `mngo/pos` )

- **Description**: The omni-channel sales point and control core.
- **Features**: Touch-optimized UI logic, quick-scan workflows, and manual price overrides.
- **Capabilities**: Support for diverse hardware (Tablets, Desktop, Handhelds).
- **Boundaries**: Handles the *orchestration of the UI action*; delegates business logic to Order/Cart.
- **How it Works**: The primary controller that aggregates data from Sales and Finance for staff interaction.
- **Use Cases**: A staff member adding 3 GA tickets and a soda to a guest's order at the front gate.
- **Integrations**: `Order`, `Payment`, `Membership`.

---

# 🛍️ SALES DOMAIN ( `packages/Mngo/Sales/` )

The conversion and revenue generation engine of the MNGO VOS.

## 44. Order Management ( `mngo/order` )

- **Description**: The definitive lifecycle manager for all customer purchases.
- **Features**: Order states (Draft, Paid, Fulfilled, Canceled), line-item tracking, and tax-inclusive totals.
- **Capabilities**: Support for partial fulfillment and complex order splits.
- **Boundaries**: Handles the *historical/finalized transaction*; temporary state is handled by Cart.
- **How it Works**: Converts a Cart into a persistent legal record once payment is confirmed.
- **Use Cases**: Tracking the status of an online order from "Payment Received" to "QR Codes Issued".
- **Integrations**: `Cart`, `Payment`, `Transaction`.

## 45. Cart Management ( `mngo/cart` )

- **Description**: Ephemeral stage for guest purchases and selection.
- **Features**: Real-time total calculation, stock-holding logic, and multi-currency support.
- **Capabilities**: Persistence across sessions (via User) and "Abandoned Cart" event emission.
- **Boundaries**: Handles *temporary selection*; does NOT handle long-term order history.
- **How it Works**: Aggregates product selections and applies real-time pricing and promotion rules.
- **Use Cases**: A guest adding tickets and a souvenir to their basket on the mobile website.
- **Integrations**: `Product`, `Pricing`, `Promotion`.

## 46. Wishlist ( `mngo/wishlist` )

- **Description**: Guest selection tool for deferred purchasing and planning.
- **Features**: Multiple wishlists per user, sharing functionality, and price-drop notifications.
- **Capabilities**: Public/Private wishlist settings and integration with CRM for marketing.
- **Boundaries**: Handles *preferences*; does NOT affect real-time inventory levels.
- **How it Works**: Stores a list of Product IDs linked to a User or Holder for future reference.
- **Use Cases**: A guest saving a "VIP Tour" to their wishlist for a visit next month.
- **Integrations**: `User`, `Product`, `Marketing/CRM`.

## 47. Bundle Engine ( `mngo/bundle` )

- **Description**: Management of complex, mix-and-match product packages.

- **Features**: Choice-based bundles (e.g., "Any 3 rides + lunch"), discount tiers, and component inventory tracking.
- **Capabilities**: Recursive bundle resolution and support for nested items.
- **Boundaries**: Handles the *composition of products*; prices are still verified via Pricing.
- **How it Works**: Expands a single bundle SKU into multiple UVTIs or line-items for fulfillment.
- **Use Cases**: Selling a "Family Fun Pack" that includes 4 tickets, 4 meals, and a parking pass.
- **Integrations**: `Product`, `Order`, `UVTI`.

## 48. Resale (P2P) ( `mngo/resale` )

- **Description**: Secure, verified marketplace for guest-to-guest ticket trading.
- **Features**: Ticket verification, automated escrow, price cap enforcement, and seller payouts.
- **Capabilities**: Fraud prevention via original UVTI invalidation and new UVTI issuance.
- **Boundaries**: Handles *peer-to-peer logic*; does NOT manage primary sales (handled by Order).
- **How it Works**: Facilitates the transfer of entitlement ownership while ensuring the venue receives its cut/fee.
- **Use Cases**: A guest selling their concert tickets to another guest because they can no longer attend.
- **Integrations**: `UVTI`, `Payment`, `Payout`.

## 49. B2B Portal ( `mngo/b2b` )

- **Description**: Management of agents, tour operators, and corporate partners.
- **Features**: Agent credit limits, tiered commission structures, and white-label agent booking portals.
- **Capabilities**: Bulk booking tools and automated commission settlement.
- **Boundaries**: Handles *partner logic*; delegates core sales to the Order module.
- **How it Works**: Provides a specialized Sales Channel for partners with specific pricing and inventory.
- **Use Cases**: Allowing a local hotel partner to book tickets for their guests at a 10% commission.
- **Integrations**: `SalesChannel`, `Payout`, `Finance`.

## 50. Storefront (Web) ( `mngo/storefront` )

- **Description**: The guest-facing B2C web engine and portal.
- **Features**: Headless commerce integration, guest profile portal, and SEO-optimized product pages.
- **Capabilities**: Ultra-fast page loads and mobile-responsive checkout flows.
- **Boundaries**: Handles the *presentation and user experience*; delegates logic to Cart/Order.
- **How it Works**: The primary interface for web browsers to interact with the MNGO ecosystem.

- **Use Cases**: The official "MNGO Park" website where guests buy their tickets.
- **Integrations**: `CMS`, `SEO`, `Cart`.

## 51. Sales Channels ( `mngo/sales-channel` )

- **Description**: Orchestration of product visibility and rules across different platforms.
- **Features**: Channel-based inventory quotas, specific pricing per channel (OTA vs. Web), and API access keys.
- **Capabilities**: High-performance "Visibility Engine" to filter product catalogs.
- **Boundaries**: Manages *channel rules*; does NOT handle the physical UI of the channel.
- **How it Works**: Validates if a product/price is valid for the current incoming source (e.g., Mobile App vs. Kiosk).
- **Use Cases**: Restricting a "Special Promo Ticket" to only be visible on the Mobile App channel.
- **Integrations**: `Product`, `Pricing`, `B2B`.

## 52. Promotion Engine ( `mngo/promotion` )

- **Description**: Management of marketing campaigns, coupons, and discount triggers.
- **Features**: Coupon code validation, automated BOGO (Buy-One-Get-One) logic, and loyalty-based discounts.
- **Capabilities**: Complex rule evaluation (e.g., "Spend > $100 and get a free hat if you are a Gold member").
- **Boundaries**: Handles *discounts*; the base product price is still defined in Pricing.
- **How it Works**: Intercepts the Cart calculation to apply "Discounts" as non-destructive line-items.
- **Use Cases**: Validating a "SUMMER20" code for a 20% discount on weekday tickets.
- **Integrations**: `Cart`, `CRM`, `Membership`.

## 53. F&B (Sales) ( `mngo/fnb` )

- **Description**: Specialized sales logic for Food and Beverage operations.
- **Features**: Menu management, modifiers (e.g., "No onions"), KDS (Kitchen Display System) routing, and table tracking.
- **Capabilities**: Support for weighted items and rapid-fire checkout for quick service.
- **Boundaries**: Handles *food-specific sales*; does NOT handle warehouse replenishment (Operations).
- **How it Works**: Extends the Order module with food-specific metadata and printer routing for kitchens.
- **Use Cases**: A waiter taking an order for a burger with extra cheese and routing it to the grill.
- **Integrations**: `Terminal`, `Order`, `Operations/Warehouse`.

## 54. Retail (Sales) ( `mngo/retail` )

- **Description**: Specialized sales logic for merchandise and shops.
- **Features**: SKU scanning, multi-buy offers (e.g., "3 for $10"), and integrated inventory lookup.
- **Capabilities**: Support for high-volume barcode scanning and rapid product lookups.
- **Boundaries**: Handles *goods sales*; does NOT handle the physical receipt of goods from suppliers.
- **How it Works**: Optimized for rapid item addition to a cart via GTIN/EAN barcodes.
- **Use Cases**: Scanning a plush toy at a gift shop and applying a member discount.
- **Integrations**: `Order`, `Promotion`, `Operations/Warehouse`.

## 55. Membership & Loyalty ( `mngo/membership` )

- **Description**: Management of guest tiers, points, and exclusive benefits.
- **Features**: Point earning/burning, tier progression, member-only pricing, and digital membership cards.
- **Capabilities**: Secure membership verification at POS and automated tier upgrades.
- **Boundaries**: Handles *status and benefits*; does NOT handle the basic User account.
- **How it Works**: Adds a layer of "Tiered Benefits" to the Holder/User profile.
- **Use Cases**: A "Platinum Member" receiving complimentary fast-track access and a 20% F&B discount.
- **Integrations**: `Pricing`, `Promotion`, `Holder`.

## 56. Signage & Menuboards ( `mngo/signage` )

- **Description**: Digital display management for menus and pricing.
- **Features**: Real-time price sync to boards, stock-out "Sold Out" overlays, and scheduled screen content.
- **Capabilities**: Support for diverse display hardware and remote screen refresh.
- **Boundaries**: Handles the *display of information*; does NOT handle the sale itself.
- **How it Works**: Pulls real-time data from F&B and Retail to update digital menu boards throughout the park.
- **Use Cases**: Automatically marking "Popcorn" as sold out on all digital screens once inventory hits zero.
- **Integrations**: `F&B`, `Retail`, `Warehouse`.

---

---

# 📣 MARKETING DOMAIN

# ( `packages/Mngo/Marketing/` )

Guest engagement, behavioral analysis, and automated communication.

## 57. Campaign Hub ( `mngo/campaigns` )

- **Description**: Central orchestration point for multi-channel marketing initiatives.
- **Features**: Campaign scheduling, budget tracking, and cross-channel attribution modeling.
- **Capabilities**: Support for phased rollouts and integration with external ad platforms (Google/TikTok).
- **Boundaries**: Manages the *strategy*; does NOT handle the physical sending of messages (Notification).
- **How it Works**: Organizes various marketing activities into unified "Campaign" entities for ROI tracking.
- **Use Cases**: Launching a "Spring Break 2026" campaign targeting multi-visit pass holders.
- **Integrations**: `CRM` , `Notification` , `Intelligence/BI` .

## 58. CRM ( `mngo/crm` )

- **Description**: Unified 360-degree behavioral guest profile repository.
- **Features**: Guest segmentation, behavioral tagging (e.g., "Frequent Diner"), and lifetime value (LTV) tracking.
- **Capabilities**: Real-time segmentation updates based on POS and Web activities.
- **Boundaries**: Handles *behavioral and marketing data*; does NOT handle physical ID (Holder) or Auth (User).
- **How it Works**: Data from Sales and Events is aggregated into the guest's CRM profile to build a "Golden Record".
- **Use Cases**: Identifying all guests who visited in the last 6 months but haven't returned.
- **Integrations**: `Holder` , `Order` , `Intelligence/Recommendation` .

## 59. Marketing Automation ( `mngo/automation` )

- **Description**: Logic engine for event-driven marketing workflows.
- **Features**: Trigger-based flows (e.g., "Abandoned Cart"), drip campaigns, and delay-based logic.
- **Capabilities**: Complex branching logic based on guest interactions.
- **Boundaries**: Handles the *logic flow*; delegates message delivery to the Notification module.
- **How it Works**: Monitors system events (e.g., `OrderCompleted` ) and triggers predefined marketing paths.
- **Use Cases**: Sending an automated "Welcome" email 24 hours after a guest registers.
- **Integrations**: `Campaigns` , `Notification` , `Sales/Cart` .

## 60. Newsletter ( `mngo/newsletter` )

- **Description**: Management of email subscriptions and bulk outbound newsletters.
- **Features**: Subscription preferences, mailing list segmentation, and template management.
- **Capabilities**: Handling double opt-in processes and automated unsubscribe logic (GDPR/Compliance).
- **Boundaries**: Handles *email marketing specific logic*; does NOT handle real-time transactional alerts (Notification).
- **How it Works**: Manages the recurring communication relationship between the venue and its subscribers.
- **Use Cases**: Sending a monthly "Park Highlights" newsletter to 50,000 subscribers.
- **Integrations**: `Notification`, `CRM`, `Ecosystem/Compliance`.

## 61. Review & Ratings ( `mngo/review` )

- **Description**: Moderated guest feedback and social proof management.
- **Features**: Star ratings, photo uploads, staff-reply workflows, and automated sentiment analysis.
- **Capabilities**: Integration with external review sites (TripAdvisor/Google) and abuse detection.
- **Boundaries**: Handles *external reviews*; does NOT handle internal NPS (deferred to Survey).
- **How it Works**: Allows guests to post feedback on products/venues, which is then moderated before public display.
- **Use Cases**: A guest rating a roller coaster 5 stars and uploading a photo of their visit.
- **Integrations**: `Product`, `Venue`, `Intelligence/AI`.

## 62. Gamification ( `mngo/gamification` )

- **Description**: Engagement engine using game-like elements (points, badges, streaks).
- **Features**: Badge issuance, leaderboard management, and achievement tracking.
- **Capabilities**: Real-time event hooks to award points based on park behaviors.
- **Boundaries**: Handles *psychological rewards*; does NOT handle financial credits (deferred to Wallet).
- **How it Works**: Tracks actions (e.g., "Visits 5 rides in a day") and grants virtual rewards/status.
- **Use Cases**: Awarding a "Weekend Warrior" badge to a guest who visits both Saturday and Sunday.
- **Integrations**: `CRM`, `Events`, `Membership`.

## 63. Survey & NPS ( `mngo/survey` )

- **Description**: Real-time guest sentiment and Net Promoter Score tracking.
- **Features**: Post-visit surveys, in-app questions, and NPS calculation dashboards.
- **Capabilities**: Conditional logic in surveys and automated follow-ups for negative feedback.
- **Boundaries**: Handles *structured feedback*; does NOT handle general social reviews (Review).
- **How it Works**: Triggers survey requests after a guest interaction (e.g., 2 hours after exit).

- **Use Cases**: Measuring guest satisfaction with the cleanliness of the venue via a 2-question survey.
- **Integrations**: `Notification`, `CRM`, `Intelligence/BI`.

## 64. Chat Interaction ( `mngo/chat` )

- **Description**: Real-time messaging hub for guest-to-staff communication.
- **Features**: Multi-channel chat (Web/App/WhatsApp), staff routing, and chat history.
- **Capabilities**: Support for media attachments and automated agent handovers.
- **Boundaries**: Handles the *communication session*; does NOT handle automated bot logic (AI Engine).
- **How it Works**: A unified inbox where guest messages from any channel are routed to available staff.
- **Use Cases**: A guest asking a live operator about locker availability via the mobile app.
- **Integrations**: `Notification`, `Intelligence/AI`, `Ecosystem/Integration`.

## 65. Notification ( `mngo/notification` )

- **Description**: Multi-channel message orchestration (SMS, Email, Push, WhatsApp).
- **Features**: Priority queuing, template localization, and delivery status tracking (DLR).
- **Capabilities**: Automated fallback (e.g., "SMS if Push fails") and rate-limiting to prevent spam.
- **Boundaries**: Handles the *transport and delivery*; does NOT hold behavioral data (CRM).
- **How it Works**: The central gateway through which all other modules send communications to guests.
- **Use Cases**: Sending a ticket QR code via email or a "Your table is ready" SMS.
- **Integrations**: Entire ecosystem, `Ecosystem/Integration`.

## 66. CMS ( `mngo/cms` )

- **Description**: Content Management System for web and app presentation.
- **Features**: Drag-and-drop page builder, block components, and multi-language content.
- **Capabilities**: Preview environments and integration with the DAM for assets.
- **Boundaries**: Handles *unstructured content (pages/blogs)*; does NOT manage structured products.
- **How it Works**: Provides a dynamic way to update "Marketing Pages" without code deployments.
- **Use Cases**: Creating a "National Day Special" landing page with banners and text.
- **Integrations**: `DAM`, `SEO`, `Storefront`.

## 67. SEO Engine ( `mngo/seo` )

- **Description**: Optimization layer for search engine visibility.

- **Features**: Automated sitemap generation, rich snippet (Schema.org) injection, and URL aliasing.
- **Capabilities**: Monitoring search rankings and suggesting keyword improvements.
- **Boundaries**: Handles *metadata and structure*; does NOT create the content (CMS).
- **How it Works**: Injects search-friendly tags and provides high-performance sitemaps for Google/Bing.
- **Use Cases**: Ensuring product pages appear with star ratings and prices in Google search results.
- **Integrations**: `CMS`, `Storefront`, `Product`.

### 68. Waiver & Liability (`mngo/waiver`)

- **Description**: Management of legal documents and digital signatures.
- **Features**: E-signature capture, document versioning, and minor/guardian linking.
- **Capabilities**: Secure PDF storage of signed waivers and real-time validity checks at gates.
- **Boundaries**: Handles *legal consent*; does NOT handle general ticketing or access control.
- **How it Works**: Requires a guest to sign a document before their UVTI becomes "Active" for certain rides.
- **Use Cases**: A parent signing a liability waiver for their child before they enter a trampoline park.
- **Integrations**: `Holder`, `UVTI`, `Legal`.

---

## 🤖 INTELLIGENCE DOMAIN (`packages/Mngo/Intelligence/`)

AI-driven insights, predictive modeling, and data-driven observability.

### 69. AI Engine (`mngo/ai`)

- **Description**: The core LLM and machine learning integration hub.
- **Features**: Natural language command processing, automated translation, and anomaly detection.
- **Capabilities**: Integration with OpenAI/Anthropic/Gemini and local model support (Llama).
- **Boundaries**: Handles *general intelligence*; specific pricing AI is handled by Price Engine.
- **How it Works**: Orchestrates requests to various AI models and interprets results for MNGO modules.
- **Use Cases**: Staff asking "How many tickets did we sell today?" via a voice command.
- **Integrations**: `Chat`, `Reporting`, `Search`.

### 70. Price Engine (AI) (`mngo/price-engine`)

- **Description**: Predictive ML for dynamic price optimization.

- **Features**: Demand forecasting, competitor price scraping, and auto-adjustment rules.
- **Capabilities**: Time-series analysis to suggest the optimal price based on weather or historical data.
- **Boundaries**: Recommends *optimal prices*; the Pricing module still holds the final settings.
- **How it Works**: Analyzes historical `Order` data to suggest price hikes or drops to maximize revenue.
- **Use Cases**: Suggesting a 10% price increase for the upcoming holiday weekend based on high demand.
- **Integrations**: `Pricing`, `Order`, `Events`.

## 71. Reporting Engine ( `mngo/reporting` )

- **Description**: Management of structured data exports and reports.
- **Features**: Scheduled PDF/Excel exports, custom report builder, and automated distribution.
- **Capabilities**: Large-scale data aggregation across multi-tenant environments.
- **Boundaries**: Generates *static reports*; interactive dashboards are handled by BI.
- **How it Works**: Queries transactional databases or data warehouses to produce formatted records.
- **Use Cases**: Automatically emailing a "Monthly Sales Summary" to the Finance director.
- **Integrations**: `Finance`, `Sales`, `Intelligence/BI`.

## 72. Business Intelligence ( `mngo/business-intelligence` )

- **Description**: Interactive real-time dashboards and data visualization.
- **Features**: Live venue telemetry dashboards, heatmaps, and multi-tenant performance comparison.
- **Capabilities**: Integration with visualization tools (Grafana/Tableau) and real-time data streaming.
- **Boundaries**: Focuses on *visual insights*; does NOT handle raw data extraction (Reporting).
- **How it Works**: Aggregates real-time streams to display the "Pulse" of the venue operations.
- **Use Cases**: A venue manager watching a live "Heatmap" of guest density to redirect staff.
- **Integrations**: `Telemetry`, `Reporting`, `Governance`.

## 73. Recommendation ( `mngo/recommendation` )

- **Description**: AI-driven up-selling and cross-selling engine.
- **Features**: "Frequently bought together" logic, personalized ride suggestions, and smart cart additives.
- **Capabilities**: Collaborative filtering models trained on global guest behavior.
- **Boundaries**: Handles *suggestions*; does NOT handle the promotion itself (Promotion Engine).
- **How it Works**: Analyzes a guest's current cart and history to suggest the next best product.
- **Use Cases**: Suggesting a "Sodexo Meal Deal" when a guest adds 2 tickets to their cart.

- **Integrations**: `Cart`, `CRM`, `Promotion`.

### 74. Search Discovery (`mngo/search`)

- **Description**: High-performance search and faceted discovery core.
- **Features**: Semantic search, autocomplete, and complex filters (e.g., "Rides for kids <100cm").
- **Capabilities**: Use of Meilisearch/Elasticsearch for ultra-fast product discovery.
- **Boundaries**: Handles *finding data*; does NOT handle the display (Storefront).
- **How it Works**: Indexes catalogs and behavioral data to provide relevant results in <50ms.
- **Use Cases**: A guest searching for "Roller Coasters Open Now" on the mobile app.
- **Integrations**: `Product`, `Venue`, `Storefront`.

### 75. Telemetry (`mngo/telemetry`)

- **Description**: Health monitoring for system performance and infrastructure.
- **Features**: Heartbeat monitoring, error rate alerts, and distributed tracing (Jaeger/Zipkin).
- **Capabilities**: Real-time alerting to Slack/PagerDuty for system degradations.
- **Boundaries**: Monitors *system health*; does NOT monitor business sales (Reporting).
- **How it Works**: Captures technical metrics (CPU, Ram, Latency, Exceptions) across all microservices.
- **Use Cases**: Alerting DevOps when the Ticket Scanning API latency exceeds 500ms.
- **Integrations**: Entire Ecosystem.

### 76. DAM (`mngo/dam`)

- **Description**: Digital Asset Management for media and files.
- **Features**: Image resizing, versioning, video transcoding, and secure CDN distribution.
- **Capabilities**: Automatic watermark injection and AI-based image tagging/filtering.
- **Boundaries**: Handles *file assets*; does NOT handle the business metadata (deferred to CMS/Product).
- **How it Works**: Central repository for all high-resolution images, videos, and PDFs used in MNGO.
- **Use Cases**: Storing the original high-res ride photos before distributing them to guests.
- **Integrations**: `Photo Engine`, `Product`, `CMS`.

---

## 🛠️ OPERATIONS DOMAIN (`packages/Mngo/Operations/`)

Material logistics, venue hardware, and human resource orchestration.

## 77. Warehouse (WMS) ( `mngo/warehouse` )

- **Description**: Inventory management and logistical floor control.
- **Features**: Stock tracking, bin locations, transfer orders, and cycle counting.
- **Capabilities**: Integration with external WMS providers (Omniful) and handheld scanner support.
- **Boundaries**: Manages *inventory stock*; does NOT handle the customer sale transaction (Sales).
- **How it Works**: The master of stock levels; every Retail/F&B sale decrements stock here.
- **Use Cases**: Receiving a shipment of 500 T-shirts and distributing them to 3 gift shops.
- **Integrations**: `Retail` , `F&B` , `Procurement` .

## 78. Procurement ( `mngo/procurement` )

- **Description**: Supply chain management and purchasing.
- **Features**: Purchase Orders (PO), supplier management, and automated replenishment alerts.
- **Capabilities**: Tracking lead times and supplier performance scores.
- **Boundaries**: Handles *buying from suppliers*; does NOT handle selling to guests.
- **How it Works**: Triggers a PO when warehouse stock falls below a predefined reorder point.
- **Use Cases**: Ordering 1,000 units of "MNGO Cola" from a supplier.
- **Integrations**: `Warehouse` , `Finance` .

## 79. Device Management ( `mngo/device` )

- **Description**: Management and monitoring of venue hardware.
- **Features**: IoT device registry, remote updates, and real-time connectivity status.
- **Capabilities**: Support for printers, turnstiles, handheld scanners, and kiosks.
- **Boundaries**: Handles the *hardware entity*; does NOT handle the business logic (AccessControl).
- **How it Works**: Maintains a persistent connection to hardware devices to monitor health and push configs.
- **Use Cases**: Remotely restarting an unresponsive self-service kiosk at the front gate.
- **Integrations**: `AccessControl` , `Terminal` , `Telemetry` .

## 80. Maintenance & Safety ( `mngo/maintenance` )

- **Description**: Management of venue upkeep and safety protocols.
- **Features**: Maintenance logs, ride safety checklists, and scheduled inspections.
- **Capabilities**: Automated ride "Down/Active" status hooks for gates.
- **Boundaries**: Handles *safety/repairs*; does NOT handle the guest experience.
- **How it Works**: Tracks the physical condition of attractions and ensures safety SOPs are followed.
- **Use Cases**: A technician completing a 50-point safety check on the Ferris Wheel.
- **Integrations**: `Venue` , `Events/Resource` , `Notification` .

## 81. Rostering ( `mngo/rostering` )

- **Description**: Staff scheduling and labor management.
- **Features**: Shift planning, employee availability, and labor law compliance engine.
- **Capabilities**: Support for dynamic staffing based on predicted crowds (from AI Engine).
- **Boundaries**: Handles *scheduling*; does NOT handle active POS sessions (Shift Management).
- **How it Works**: Assigns Holders (Staff) to specific Roles and Departments for set time blocks.
- **Use Cases**: Scheduling 20 cashiers for the busy Saturday shift.
- **Integrations**: `Identity`, `Department`, `Intelligence/AI`.

## 82. Support & Tickets ( `mngo/support` )

- **Description**: Internal and external helpdesk and SLA tracking.
- **Features**: Ticket creation, SLA escalations, and internal knowledge base.
- **Capabilities**: Integration with guest channels (WhatsApp/Email) to convert chats to tickets.
- **Boundaries**: Handles *support lifecycle*; does NOT handle the real-time chat (deferred to Chat).
- **How it Works**: Tracks issues from guests or staff to ensure they are resolved within set SLAs.
- **Use Cases**: A guest filing a ticket about a lost item in the park.
- **Integrations**: `Chat`, `Notification`, `Events`.

## 83. Hotel Management ( `mngo/hotel` )

- **Description**: Integrated resort charging and PMS synchronization.
- **Features**: Room charging, guest status sync, and hotel-guest perk management.
- **Capabilities**: Integration with external PMS systems (Oracle Opera, Protel).
- **Boundaries**: Handles *resort-wide charging*; does NOT handle hotel room cleaning or PMS core logic.
- **How it Works**: Allows hotel guests to "charge to room" at any park POS using their room key/wristband.
- **Use Cases**: A hotel guest paying for dinner at the park using their room number for verification.
- **Integrations**: `Payment`, `Wallet`, `Media Management`.

## 84. Parking & Valet ( `mngo/parking` )

- **Description**: Vehicle access control and space utilization tracking.
- **Features**: Plate recognition (LPR) integration, valet ticket tracking, and parking space sensors.
- **Capabilities**: Dynamic parking rates based on occupancy levels.
- **Boundaries**: Handles *vehicle logistics*; does NOT handle general guest entry (AccessControl).
- **How it Works**: Manages the entry and exit of vehicles, linking them to a `UVTI` or `Order`.
- **Use Cases**: Scanning a parking QR code at the gate to automatically open the barrier.

- **Integrations**: `AccessControl`, `Order`, `Terminal`.

### 85. Lockers & Rentals (`mngo/rental`)

- **Description**: Management of duration-based rental assets.
- **Features**: Locker assignment, rental duration tracking, and automated overdue alerts.
- **Capabilities**: Integration with smart electronic lockers and stroller/wheelchair inventory.
- **Boundaries**: Handles *temporary asset use*; does NOT handle permanent sales.
- **How it Works**: Issues a short-term entitlement to a physical asset (Locker 42) for a set time.
- **Use Cases**: A guest renting a locker for 4 hours and receiving a digital key on their app.
- **Integrations**: `Product`, `Entitlement`, `Order`.

### 86. Operations Hub (`mngo/operations`)

- **Description**: Central dashboard for command and control of venue ops.
- **Features**: Real-time KPI wallboards, crisis management toggles, and staff alerts.
- **Capabilities**: Aggregated view of all active shifts, queues, and hardware health.
- **Boundaries**: Focuses on *aggregated visibility*; does NOT implement the sub-modules' logic.
- **How it Works**: The master "Battle for Control" dashboard for venue directors.
- **Use Cases**: A park director monitoring total gate entry counts vs. F&B sales in real-time.
- **Integrations**: Entire Operations domain, `Intelligence/BI`.

---

---

## 🌐 ECOSYSTEM DOMAIN (`packages/Mngo/Ecosystem/`)

Global infrastructure, integration bridges, and platform-wide foundations.

### 87. ApiGateway (NestJS) (`mngo/api-gateway`)

- **Description**: High-performance entry point and request orchestrator for the MNGO microservices.
- **Features**: Request routing, rate-limiting, centralized logging, and protocol translation (REST/gRPC).
- **Capabilities**: Support for WebSockets and real-time event streaming via NestJS microservices.
- **Boundaries**: Handles *perimeter logic*; does NOT implement business rules.
- **How it Works**: Acts as the single "Front Door" for all clients (Web/Mobile/IoT).

- **Use Cases**: Routing a guest's profile request to the `Holder` module after validating their `Auth` token.
- **Integrations**: Entire ecosystem.

## 88. Geography (Info) ( `mngo/geography` )

- **Description**: Static and dynamic data about the physical world.
- **Features**: ISO country codes, city databases, timezones, and currency metadata.
- **Capabilities**: High-speed lookup for regional settings and address formatting rules.
- **Boundaries**: Handles *world geography facts*; does NOT handle business regions (Region).
- **How it Works**: Provides a standardized set of geographical data for use across all venues.
- **Use Cases**: Populating a "Country" dropdown menu or calculating a timezone difference.
- **Integrations**: `Holder`, `Region`, `Tax Engine`.

## 89. Integration Proxy ( `mngo/integration` )

- **Description**: Standardized bridge for third-party external services.
- **Features**: Provider abstraction (e.g., "Generic SMS Provider"), retry logic, and payload transformation.
- **Capabilities**: High-performance proxying for external APIs like Stripe, Twilio, or Oracle.
- **Boundaries**: Handles the *connection to the outside world*; does NOT handle internal MNGO logic.
- **How it Works**: Wraps external SDKs in a consistent MNGO-compatible interface.
- **Use Cases**: Switching from Twilio to another SMS provider without changing business logic code.
- **Integrations**: `Payment`, `Notification`, `Operations/Hotel`.

## 90. Developer Portal ( `mngo/developer-portal` )

- **Description**: Self-service center for staff and third-party developers.
- **Features**: API documentation (Swagger/OpenAPI), API key management, and application registry.
- **Capabilities**: Interactive "Try-it-out" consoles and SDK generation.
- **Boundaries**: Handles *developer experience*; does NOT handle the core API logic.
- **How it Works**: Serves as the community and technical hub for extending the MNGO platform.
- **Use Cases**: A third-party developer registering an app to display live park wait times.
- **Integrations**: `ApiGateway`, `Identity/RBAC`, `Sandbox`.

## 91. Sandbox Environment ( `mngo/sandbox` )

- **Description**: Isolated testing and development environment management.
- **Features**: Mock data generation, isolated database tenants, and simulated execution paths.
- **Capabilities**: "Time-travel" testing (simulating future visit dates) in an isolated scope.

- **Boundaries**: Handles *isolated testing*; should never touch production data.
- **How it Works**: Provides a walled-off version of the ecosystem for safe experimentation.
- **Use Cases**: A developer testing a new pricing rule without affecting real park sales.
- **Integrations**: Entire ecosystem.

## 92. Batch Processor ( `mngo/batch` )

- **Description**: High-volume background task automation engine.
- **Features**: Job scheduling, queue management, and long-running process monitoring.
- **Capabilities**: Distribution of heavy tasks (e.g., massive reporting) across worker nodes.
- **Boundaries**: Handles the *execution of background work*; does NOT handle real-time user requests.
- **How it Works**: Pulls jobs from a persistent queue (Redis/DB) and executes them asynchronously.
- **Use Cases**: Processing 100,000 monthly statements on the 1st of every month.
- **Integrations**: `Reporting` , `Notification` , `Finance` .

## 93. Url Redirects ( `mngo/url` )

- **Description**: Management of SEO-friendly aliases and legacy URL redirection.
- **Features**: 301/302 redirect management, vanity URL creation, and campaign tracking links.
- **Capabilities**: High-speed redirect lookup and monitoring of "404 Not Found" errors.
- **Boundaries**: Handles *URI routing logic*; does NOT handle the page content (CMS).
- **How it Works**: Intercepts requests for shortened or legacy URLs and routes them to new destinations.
- **Use Cases**: Redirecting `mngo.io/discounts` to a complex promotional landing page.
- **Integrations**: `CMS` , `SEO` , `Storefront` .

## 94. Legal & GDPR ( `mngo/legal` )

- **Description**: Compliance management for privacy and data protection.
- **Features**: Consent logging, "Right to be Forgotten" workflows, and privacy policy versioning.
- **Capabilities**: Automated data anonymization and user data export tools.
- **Boundaries**: Handles *data privacy logic*; does NOT handle physical staff HR legalities.
- **How it Works**: Tracks when and where a guest gave consent for data processing.
- **Use Cases**: Processing a guest's request to delete all their behavioral history from the CRM.
- **Integrations**: `Holder` , `CRM` , `Audit` .

## 95. Compliance ( `mngo/compliance` )

- **Description**: Internal controls and external standards enforcement (PCI-DSS, SOC2).
- **Features**: Compliance checks, security posture reporting, and policy enforcement.

- **Capabilities**: Automated scanning for sensitive data (e.g., CC numbers) in non-secure logs.
- **Boundaries**: Monitors *compliance status*; does NOT implement the security patches.
- **How it Works**: Audits the system against industry standards to ensure legal operation.
- **Use Cases**: Verifying that no full credit card numbers are stored in the `Audit` logs.
- **Integrations**: `Audit`, `Finance`, `Security`.

## 96. Security ( `mngo/security` )

- **Description**: Perimeter defense and internal data encryption.
- **Features**: WAF (Web Application Firewall) integration, rate-limiting, and encryption-at-rest management.
- **Capabilities**: Intrusion detection and automated IP blacklisting for brute-force attacks.
- **Boundaries**: Handles the *technical security layer*; does NOT handle business logic permissions (RBAC).
- **How it Works**: Protects the MNGO ecosystem from technical threats and ensures data is encrypted.
- **Use Cases**: Automatically blocking an IP address after 50 failed login attempts.
- **Integrations**: `Auth`, `ApiGateway`, `Telemetry`.

## 97. Backup/Recovery ( `mngo/backup` )

- **Description**: Disaster recovery and data redundancy management.
- **Features**: Automated snapshotting, point-in-time recovery, and cross-region replication.
- **Capabilities**: Verification of backup integrity and automated failover drills.
- **Boundaries**: Handles *data persistence safety*; does NOT handle real-time sync (Sync).
- **How it Works**: Ensures that in the event of a catastrophic failure, the VOS can be restored.
- **Use Cases**: Restoring the database to its state exactly 5 minutes before a corruption event.
- **Integrations**: Entire Ecosystem.

## 98. Deployment ( `mngo/deployment` )

- **Description**: Orchestration of the software delivery lifecycle.
- **Features**: K8s manifest management, blue-green deployment logic, and environment configuration.
- **Capabilities**: Zero-downtime updates for high-availability venue services.
- **Boundaries**: Handles *how code runs*; does NOT handle the code's business logic.
- **How it Works**: Standardizes how MNGO modules are packaged, versioned, and deployed to infra.
- **Use Cases**: Deploying a new version of the `Pricing` module to a specific park without downtime.
- **Integrations**: `Telemetry`, `Developer Portal`.

## 99. Mobile Guest ( `mngo/mobile-guest` )

- **Description**: Core API and logic specifically for guest-facing mobile applications.
- **Features**: Deep-linking, mobile-optimized checkout, and native push integration.
- **Capabilities**: Handling sporadic connectivity and mobile security tokens.
- **Boundaries**: Handles *mobile guest business logic*; does NOT handle the UI components (Frontend).
- **How it Works**: Provides a specialized set of APIs tailored for the low-bandwidth mobile experience.
- **Use Cases**: A guest viewing their "Digital Ticket Wallet" while walking through the park entry gates.
- **Integrations**: `Storefront` , `Cart` , `Notification` .

## 100. Mobile Staff ( `mngo/mobile-staff` )

- **Description**: Core API and logic specifically for staff-facing handheld devices.
- **Features**: High-speed scanning logic, staff shift controls, and task management.
- **Capabilities**: Support for specialized hardware (Zebra/Honeywell) and offline staff actions.
- **Boundaries**: Handles *mobile staff business logic*; does NOT handle general guest interactions.
- **How it Works**: Optimized for rapid operational tasks like entry scanning or incident reporting.
- **Use Cases**: A staff member scanning a guest's wristband on the platform to verify ride entry.
- **Integrations**: `AccessControl` , `Terminal` , `Shift` .

# 🎯 TOTAL MODULE COUNT: 100 Core Modules

The architecture is now optimized for the **MNGO VOS** (Venue Operating System), ensuring zero redundancy, perfect domain isolation, and absolute architectural clarity. 🥭🚀🏆💎⛰️